RTL DESIGN OF FINITE STATE MACHINES (FSM) AND CONTROL UNITS

VERILOG HDL SOURCE CODES AND TESTBENCHES

VLSI DESIGN INTERNSHIP – TASK 4

Submitted By;

Sowjanya Gollapalli

Company:

MAINCRAFTS TECHNOLOGY

Internship Duration:

May 20 to July 5

1. MOORE FSM

Verilog Code:

```
module moore_fsm(
   input clk,
   input rst,
   input in,
   output reg [1:0] out
);
reg [1:0] state, next_state;
parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10;
always @(posedge clk or posedge rst)
begin
```

```
    if(rst)
        state <= S0;
    else
        state <= next_state;
end
always @(*)
begin
    case(state)
        S0: next_state = in ? S1 : S0;
        S1: next_state = in ? S2 : S1;
        S2: next_state = in ? S0 : S2;
        default: next_state = S0;
    endcase
end
always @(*)
begin
    case(state)
        S0: out = 2'b00;
        S1: out = 2'b01;
        S2: out = 2'b10;
        default: out = 2'b00;
    endcase
end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb_moore_fsm;
reg clk;
reg rst;
reg in;
wire [1:0] out;
moore_fsm uut(
    .clk(clk),
    .rst(rst),
    .in(in),
    .out(out)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    in = 0;
    #10 rst = 0;
    in = 1; #20;
    in = 1; #20;
    in = 0; #20;
    in = 1; #20;
    #20;
    $finish;
```

```
end

endmodule
```

2. MEALY FSM

Verilog Code:

```
module mealy_fsm(
   input clk,
   input rst,
   input din,
   output reg z
);
reg [1:0] state, next_state;
parameter S0=2'b00,
          S1=2'b01,
          S2=2'b10;
always @(posedge clk or posedge rst)
begin
   if(rst)
      state <= S0;
   else
      state <= next_state;
end
always @(*)
begin
   z = 0;
```

```
case(state)
S0:
begin
   if(din)
      next_state = S1;
   else
      next_state = S0;
end
S1:
begin
   if(din)
      next_state = S2;
   else
      next_state = S0;
end
S2:
begin
   if(din)
   begin
      next_state = S0;
      z = 1;
   end
   else
      next_state = S1;
```

```
    end
    default:
        next_state = S0;
    endcase
end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb_mealy_fsm;
reg clk;
reg rst;
reg din;
wire z;
mealy_fsm uut(
    .clk(clk),
    .rst(rst),
    .din(din),
    .z(z)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    din = 0;
```

```
    #10 rst = 0;
    din = 1; #10;
    din = 1; #10;
    din = 1; #10;
    din = 0; #10;
    din = 1; #10;
    #20;
    $finish;
end
endmodule
```

3. TRAFFIC LIGHT CONTROLLER (MOORE FSM)

Verilog Code:

```
module traffic_light_controller(
    input clk,
    input rst,
    output reg red,
    output reg yellow,
    output reg green
);
reg [1:0] state, next_state;
parameter RED    = 2'b00;
parameter YELLOW = 2'b01;
parameter GREEN  = 2'b10;
```

```
always @(posedge clk or posedge rst)
begin
   if(rst)
      state <= RED;
   else
      state <= next_state;
end
always @(*)
begin
   case(state)
      RED:    next_state = YELLOW;
      YELLOW: next_state = GREEN;
      GREEN:  next_state = RED;
      default: next_state = RED;
   endcase
end
always @(*)
begin
   red = 0;
   yellow = 0;
   green = 0;
   case(state)
      RED: red = 1;
      YELLOW: yellow = 1;
      GREEN: green = 1;
```

```
    endcase
end
endmodule
```

Testbench

Verilog

```
`timescale 1ns/1ps
module tb_traffic_light;
reg clk;
reg rst;
wire red;
wire yellow;
wire green;
traffic_light_controller uut(
    .clk(clk),
    .rst(rst),
    .red(red),
    .yellow(yellow),
    .green(green)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #10 rst = 0;
```

```
    #100;
    $finish;
end
endmodule
```

4. SEQUENCE DETECTOR FOR 1011 (MEALY FSM)

Verilog Code:

```
module sequence_detector(
    input clk,
    input rst,
    input din,
    output reg z
);
reg [1:0] state, next_state;
parameter S0 = 2'b00;
parameter S1 = 2'b01;
parameter S2 = 2'b10;
parameter S3 = 2'b11;
always @(posedge clk or posedge rst)
begin
    if(rst)
        state <= S0;
    else
        state <= next_state;
end
```

```
always @(*)
begin
   z = 0;
   case(state)
   S0:
      if(din)
         next_state = S1;
      else
         next_state = S0;
   S1:
      if(din)
         next_state = S1;
      else
         next_state = S2;
   S2:
      if(din)
         next_state = S3;
      else
         next_state = S0;
   S3:
   begin
      if(din)
      begin
         next_state = S1;
         z = 1;
```

```
        end
        else
            next_state = S2;
    end
    default:
        next_state = S0;
    endcase
end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb_seq_detector;
reg clk;
reg rst;
reg din;
wire z;
sequence_detector uut(
    .clk(clk),
    .rst(rst),
    .din(din),
    .z(z)
);
always #5 clk = ~clk;
```

```
initial begin
    clk = 0;
    rst = 1;
    din = 0;
    #10 rst = 0;
    din = 1; #10;
    din = 0; #10;
    din = 1; #10;
    din = 1; #10;
    din = 0; #10;
    din = 1; #10;
    din = 0; #10;
    din = 1; #10;
    din = 1; #10;
    #20;
    $finish;
end
endmodule
```